

[illegible]

Design: I built a python script that uses AES encryption in Counter mode to encrypt the read command. Then, I created a relay\_modify function to calculate the bitwise difference between read and save in order to apply the difference onto the encrypted message directly. Additionally, I created a receiver\_process function that decrypts and executes the message that function receives, which then sends the same encrypted message through the receiver function twice.

Experiment Results: The receiver was able to decrypt the message and executed the save command instead of the read command without having any errors or modifications of the message. Likewise, the receiver authorized and ran the duplicate encrypted message two times without recognizing the receiver was seeing a replayed message.

Root Cause: AES in Counter mode encryption guarantees the confidentiality for security. It doesn't guarantee the maintaining integrity of the message such as preventing changes for security. This encryption method works by scrambling the text using XOR with a key stream, and since there is no digital signature attached to the message to see whether the message was modified, it is possible to change the bytes in the encrypted message in order to change the decrypted text in a more predictable manner.

## Task 2 - Diffie-Hellman setup

```
(.venv) avaayyaduraitamu@ubuntu24:~/Downloads/HW1/HW1/csce465-agentsec/hw2$ python handshake.py
=== Task 2 Handshake ===
Protocol: CSCE465-HS-v2
Group: ffdhe3072
Gateway identity: gateway
Node identity: node
Transcript length: 865
Transcript hash: 0eee4152e0706a20d5945ea8c3bbdfcceb541b00af814881a423589903b016a
Gateway signature: VALID
Node signature: VALID
Shared secret matches: True
Derived keys match: True
K_master: f613ef61a8b8872853af71a2c73d43e0b4672c1e7fe9f66ac2a56a28c4847b34
K_g2n_enc: fdad1f6cdd389699c1bf7a8efd8e4d414f8caf189b6adc7e6b99c140036a2c8e
K_g2n_mac: 403222069065159dd7bd40c585dbe5473dcc06bb76644ebfe5b65be559abd40e
K_n2g_enc: fc6015c4a0f86ad5d4c18884fba8e7cca7ec2bdd12e6ca611b1568fedf1bc26b
K_n2g_mac: 71d383ca4f058c1bd4c3cc6c4c83a3a860edb1a2359dd4ed78c0c197eb986ed1
session_id: ea0b88f8c88f60d1
Handshake: SUCCESS

=== Rejection Tests ===
Invalid signature: REJECTED
Changed nonce: REJECTED
Changed public value: REJECTED
Unexpected identity: REJECTED
Malformed transcript: REJECTED
Bad declared length: REJECTED
Reflected handshake: REJECTED
(.venv) avaayyaduraitamu@ubuntu24:~/Downloads/HW1/HW1/csce465-agentsec/hw2$
```

Diffie-Hellman (DH) Key establishment: Allows two parties to calculate a shared secret over an insecure channel using public values. Using ephemeral exponents allows a new key to be generated for every session.

Digital Signatures: Helps with the integrity part of security where signing messages with a private key is used for authentication and can be verified with the corresponding public key

Entity Authentication: Confirms operational identity of a communicating party during a live interaction

Nonces: Number values used one time per session to make sure each session is unique

Replay Resistance: a protective mechanism to guarantee that past protocol session that were valid cannot be used again

Forward secrecy: Ensures that compromising a long-term private signing key in the future will not compromise past session keys since ephemeral keys are deleted after deriving the keys and cannot be recalculated in the future

Key Separation: Using distinct separate keys in cryptographic way for different operations such as using long-term signing keys versus short-term session encryption keys

Diffie-Hellman Values prevent Middle of the Man attacks by using digital signatures that signs sender's Diffie-Hellman public key using the private signing key. Since the hacker does not have the sender private signing key, the hacker can't use a valid digital signature for fake key. As a result, when the receiver verifies the digital signature using the sender's public key, and any fake key value will be quickly discovered, and the receiver's computer will stop the handshake and end the network connection.

Identities of the sender and receiver prevent Unknown Key shake attacks and Misbinding since the digital signature binds both parties' identities directly to the key value being sent.

Roles prevent reflection attacks because digital signatures include explicit role indicators that bind the message to the sender's role. If an attacker reflects the sender's message back to the sender, the receiver's role check fails because an endpoint acting as an initiator will only accept incoming messages marked for a responder

Nonces prevent replay attacks since digital signatures bind to one time, unique number randomly generated for that one session. If the hacker attempts to replay a recorded digital signature from a past exchange, the check fails for nonces since the old message contains an expired number.

When digital signatures are removed, there is no authentication to prove who created the message. Additionally, this can cause a Denial of Service attack if there are no digital signatures since this will overuse the CPU in order to do a lot of calculations for every fake key that is being created which causes the server to not be responsive.

### Task 3- Encrypt-then-MAC record layer

```
(.venv) avaayyaduraitamu@ubuntu24:~/Downloads/HW1/HW1/csce465-agentsec/hw2$ python secure_record.py
=== Task 3 Encrypt-Then-MAC Record Layer ===

1. Normal Gateway -> Node record
Sequence used: 0
Plaintext recovered: {"action":"READ","path":"notes.txt"}
Normal record: SUCCESS

2. Second record / sequence progression
Sequence used: 1
Plaintext recovered: {"action":"WRITE","path":"notes.txt"}
Sequence 0 -> 1: SUCCESS

3. Replay
Replay of sequence 0: REJECTED

4. Modified header
Modified header: REJECTED

5. Modified ciphertext
Modified ciphertext: REJECTED

6. Invalid MAC
Invalid MAC: REJECTED

7. Wrong direction
Wrong direction: REJECTED

8. Wrong message type
Wrong message type: REJECTED

9. Wrong sequence number
Unexpected sequence: REJECTED

10. Malformed record
Malformed record: REJECTED

11. Node -> Gateway separate direction/key test
Node -> Gateway sequence: 0
Plaintext recovered: {"action":"READ","path":"notes.txt"}
Separate direction keys: SUCCESS

=== Task 3 Test Result ===
All tests passed: True
Task 3 test suite: SUCCESS
(.venv) avaayyaduraitamu@ubuntu24:~/Downloads/HW1/HW1/csce465-agentsec/hw2$
```

The Secure Record Layer is implemented as an Encrypt-then-MAC record protocol using AES-256-CTR for encryption and HMAC-SHA256 for authentication.

## Task 4-Adversarial tests and security note

```
(.venv) avaayyaduraitamu@ubuntu24:~/Downloads/HW1/HW1/csce465-agentsec/hw2$ PYTHONPATH=. pytest -v tests
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /home/avaayyaduraitamu/Downloads/HW1/HW1/csce465-agentsec/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/avaayyaduraitamu/Downloads/HW1/HW1/csce465-agentsec/hw2
collected 6 items

tests/test_modified_ciphertext.py::test_modified_ciphertext_is_rejected PASSED
tests/test_modified_header.py::test_modified_authenticated_header_is_rejected PASSED
tests/test_reflected_handshake.py::test_reflected_handshake_signature_is_rejected PASSED
tests/test_reflected_record.py::test_reflected_record_is_rejected PASSED
tests/test_replay.py::test_replayed_record_is_rejected PASSED
tests/test_valid_handshake.py::test_valid_handshake_and_bidirectional_messages PASSED

===== 6 passed in 3.77s =====
(.venv) avaayyaduraitamu@ubuntu24:~/Downloads/HW1/HW1/csce465-agentsec/hw2$
```

**Attacker Model:** There is an active network adversary that can delete, intercept, delete, replay, reorder, and inject network packets

### Security Goals:

- **Confidentiality:** Guarantees that eavesdropping adversary cannot obtain plaintext from transmitted encrypted text
- **Integrity:** Guarantees that unauthorized users cannot tamper with headers, encrypted text/payload or modify records

### Non-Goals:

- **Analyzing Network Traffic:** Information leaks from packet sizes are being mitigated as well as excluding transmission timing and network communication patterns
- **Application Layer Validation:** Protects against what seems valid but malicious instructions from the application layer of the OSI model

**Stream Cipher Risks:** Counter mode in encryption generates a key stream by combining the key with a starting number and a counter. Then, this will encrypt the message by integrating the key stream with the plaintext. If the same starting number under the same key is reused, then the system will create the same key stream. The hacker who maliciously gets two messages with the exact key stream can decrypt with using encryption patterns. In addition, counter mode does not guarantee the integrity aspect of security since changing the bytes in an encrypted message can also change the plaintext without any trigger. With this, using a unique initialization vector will prevent from using the same key stream, which will help with the confidentiality aspect of security.

Order of Cryptographic Operations: With the Encrypt-then-MAC approach, it generates an authentication tag over the encrypted message. Then, the receiver will confirm the authentication tag before decrypting the message

- Stopping Denial of Service Attacks: By allowing to confirm the authentication tag will prevent the systems from using a lot processing power or allocating memory on untrusted data by a hacker
- Stopping Oracle Attacks: When the authentication tag is required, the server validates the authentication tag on the encrypted message, and if the attacker changed the bytes of message, then the tag check fails. As a result, the server prompts to reject before decrypting the message

Independent Encryption and MAC Keys:

- Limits Damage: Keeping the keys separate guarantees that a compromise in one algorithm will not expose the other algorithms, meaning that if a hacker attempts to change the encryption key, they cannot modify authenticated messages
- Stops Algorithms from Interfering: Even though different cryptographic algorithms process data using distinct mathematical operations, reusing the same key creates vulnerabilities in a structural sense and data leaks. Attackers can exploit these leaks to bypass security assumptions, ultimately cracking the encrypted message

| Feature / Metric    | Encrypt-then-MAC                                                                               | AES-GCM                                                                                                       |
|---------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Algorithms          | Two separate algorithms                                                                        | Single unified algorithm                                                                                      |
| Processing Passes   | Requires two distinct passes: one to encrypt and one to authenticate                           | In a single integrated pass, it does encryption and authentication                                            |
| Implementation Risk | Higher chance of implementation risk due to wrong order of operation (cryptographic algorithm) | Lower chance of implementation risk due to built in functions (e.g APIs) that have the security steps handled |

With a valid protected record, there still can a dangerous authorized tool call since this only checks if the message came from a authorized user, but it does not check if the command is not malicious. For example, a hacker can steal an authorized key, but uses prompt injection, and the message would be transmitted since it has valid digital signature and key.